Randomized quick sort

- Randomized quick sort chooses a random element as a pivot.
- It is done so as to avoid the worst case of quick sort in which the input array is already sorted.
- Randomized quick sort helps in avoiding the worst case time complexity of $O(n^2)$ which occurs in case when the input array is already sorted.
- However the average case and best case time complexities remain unaltered.

Randomized Quick Sort

1. Pick a pivot element uniformly at random from the array
2. Split array into 3 subarrays: those smaller than pivot, those larger than pivot, and the pivot itself.
3. Recursively sort the subarrays, and concatenate them.

- Randomized QuickSort sorts a given array of length n in O(n log n) expected time.
- On every input randomized QuickSort takes O(n log n) time in expectation.

- On every input it may take

- $\Omega(n^2)$ time with some small probability

ALGORITHM

```
RANDOMIZED_PARTITION (A, p, r)
 i ← RANDOM (p, r)
 Exchange A[p] ↔ A[i]
 return PARTITION (A, p, r)

RANDOMIZED_QUICKSORT (A, p, r)
If p < r then
q ← RANDOMIZED_PARTITION (A, p, r)
 RANDOMIZED_QUICKSORT (A, p, q)
 RANDOMIZED_QUICKSORT (A, q+1, r)
```

- Assume that procedure Random (a, b) returns a random integer in the range [a, b); there are b-a+1 integers in the range and procedure is equally likely to return one of them.

- The new partition procedure, simply implemented the swap before actually partitioning.
- Like other randomized algorithms, RANDOMIZED_QUICKSORT has the property that no particular input elicits its worst-case behavior.
- The behavior of algorithm only depends on the random-number generator.